

SQL in Python

1. Python SQL Libraries
2. mysql-connector-python
3. Connecting to a server
4. The cursor object
5. Database connection
6. SELECT
7. CREATE TABLE
8. INSERT INTO

Python SQL Libraries

There are a multitude of SQL libraries available for Python:

- ▶ SQLite (sqlite3)
- ▶ MySQL (mysql-connector-python)
- ▶ PostgreSQL (psycopg2)
- ▶ PyPi (python-sql)
- ▶ SQLAlchemy
- ▶ Records
- ▶ PugSQL
- ▶ DjangoORM
- ▶ Peewee
- ▶ PonyORM
- ▶ SQLAlchemy
- ▶ PyMySQL
- ▶ MySQLDB
- ▶ MySQLClient
- ▶ OurSQL

- The main job of these libraries is to give you access to a way to communicate with your database
- This is done by creating connections to the server of your database or in some cases to the database on your hard drive
- Than you hand over (non-)standard SQL statements to the connection so that they can be executed
- Therefore in most cases it's more important to remember the correct SQL syntax

mysql-connector-python

- As we saw earlier the module mysql-connector-python is one of the many libraries we can use to work on SQL databases with the help of python scripts
- This module allows us to access our MySQL-databases, for example the ones we used in our Workbench, and run any kind of statement on them
- While working with this module, we can see all changes we make to our databases in our workbench as well, assuming we are using the same connection

- After we installed the module we need to import it to our python scripts to access all needed functions for working with databases
- The import is relatively simple:
 - `import mysql.connector`
- Afterwards we can use this module to connect to a specific MySQL server, be it our local one or remote ones

Connecting to a server

- The first step when working with databases in Python, is to connect to the corresponding server, so that you are able access your database(s)
- To do so we have to create a MySQLConnection object with the help of the connect function of the mysql-connector-python module:

```
connection = mysql.connector.connect(...)
```

- This function always needs three arguments to be able to connect to the corresponding server:
 1. host
 2. user
 3. password

- All arguments need to be associated with values in the form of strings:

```
connection = mysql.connector.connect(host='localhost',  
                                     user='root',  
                                     password='Snf31J,wiiK47iMS!')
```

- The argument host is associated either with the IP-address of your server or the keyword 'localhost' if you want to connect to your local MySQL server
- Since we want to work with the databases we used during the SQL part in our Workbench we need to use the 'localhost' here

- The argument user has to be associated with the username you use when working with your database
- Per default this username is 'root'
- The last argument password is now the password you need when you try to access your server
- In our case this is the password you chose when installing the MySQL server locally on your PC, again the one you use when working with the MySQL workbench

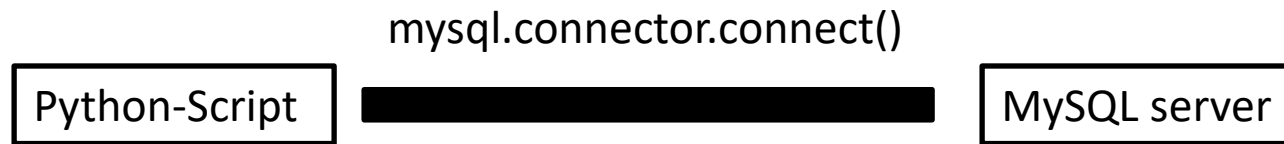
- After you're finished with your script you should close the connection to your server to avoid any security risks:

```
connection.close()
```

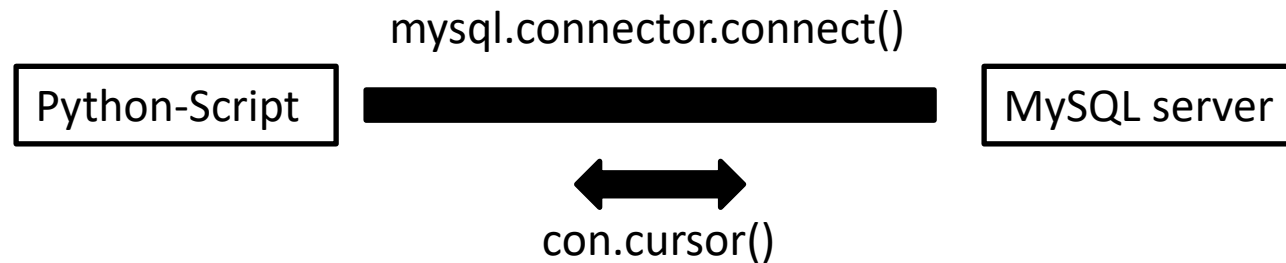
- This may not be important when working with a local server, but with remote servers it could lead to troubles if you leave such a connection open afterwards

The cursor object

- Once your connection to the server was established you need to create a cursor object on it to hand over MySQL statements to your database
- We can think of the connection we created earlier as some form of tunnel that goes from our Python script to our server



- Now we need the cursor as carrier for our statements as well as the answers from the database



- We can do this with the `cursor()` function of our `MySQLConnection` object we created earlier:

```
cursor = connection.cursor()
```

- Similar to the connection, you also need to close the cursor at the end of your script to avoid any security risks
- Keep in mind to close the cursor before you close the connection
- Again we do this with the `close()` function of our cursor object:

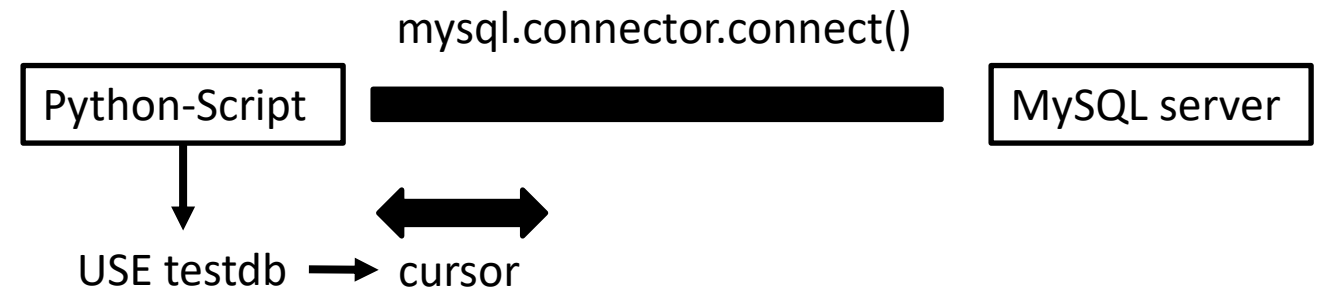
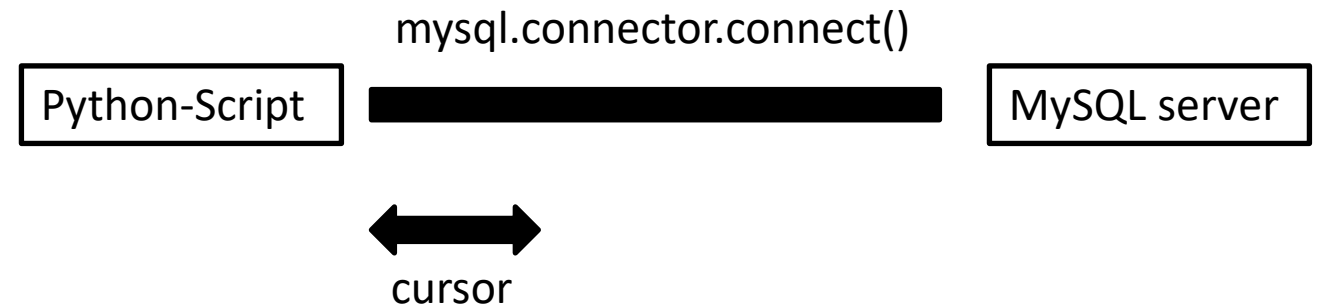
```
cursor.close()
```

Database Connection

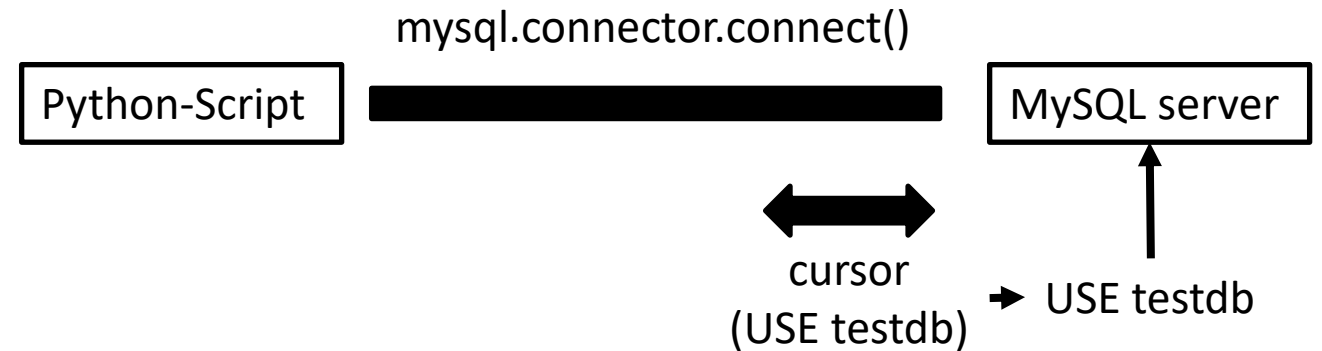
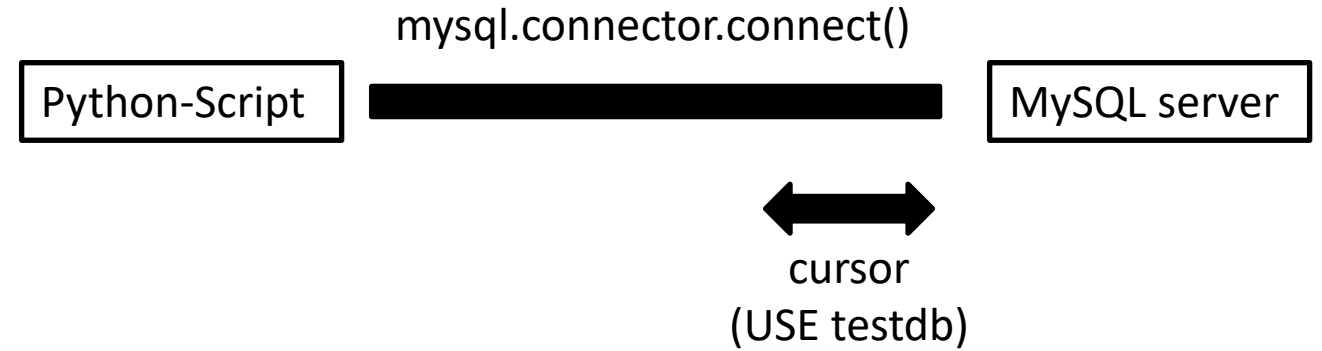
- When working with our workbench we learned that we always need to have a certain database active
- The same holds true here as well, since our cursor object needs to know to which database he needs to go
- There are two ways of connecting to a specific database
- First we can work with the statement USE and the execute function of our cursor:

```
cursor.execute('USE testdb')
```

- We saw that our connection is the tunnel to our database and the cursor a form of carrier
- With the help of the execute() function of our cursor we are able to hand over SQL statements to it to carry them to the server



- Now our cursor carries the SQL statement to the server
- Once it reaches the server, it hands over the SQL statement to it and the server tries to execute the statement



- Instead of working with the USE statement we can also connect to a database directly when creating our MySqlConnection object
- We simply need to add the argument database to it and associate the name of one the databases available on the server to it:

```
connection = mysql.connector.connect(host='localhost',  
                                     database='testdb',  
                                     user='root',  
                                     password='Snf31J,wiiK47iMS!')
```

SELECT

- Once we established a connection to our server, created a cursor object and specified a database we want to work on, we can create for example SELECT statements to read out data from our database
- We can do this again with the help of the execute() function of our cursor object:

```
cursor.execute('SELECT * FROM kingdoms')
```

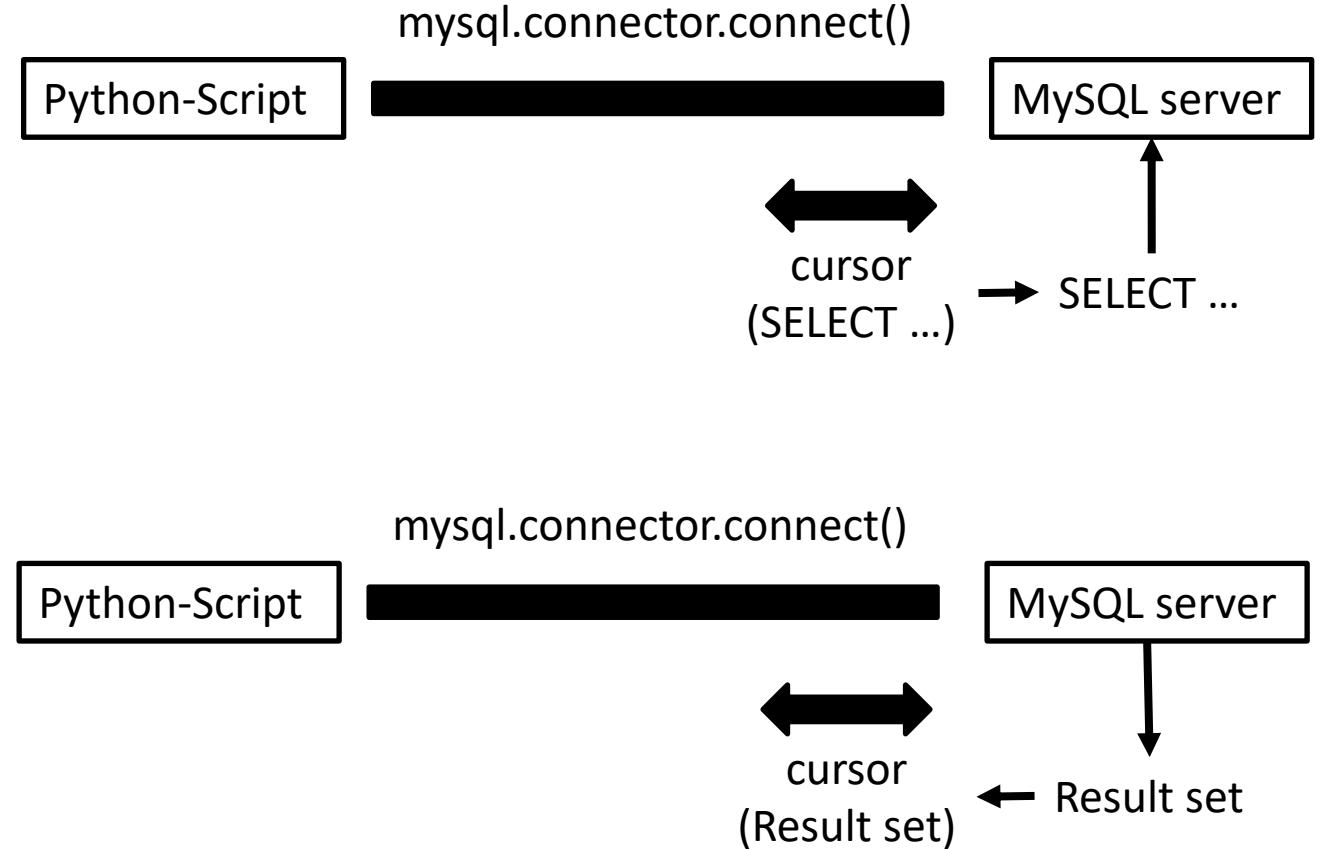
- To access the answer of the database we afterwards need to use the fetchall() function of our cursor and assign the value to a variable for example:

```
records = cursor.fetchall()
```

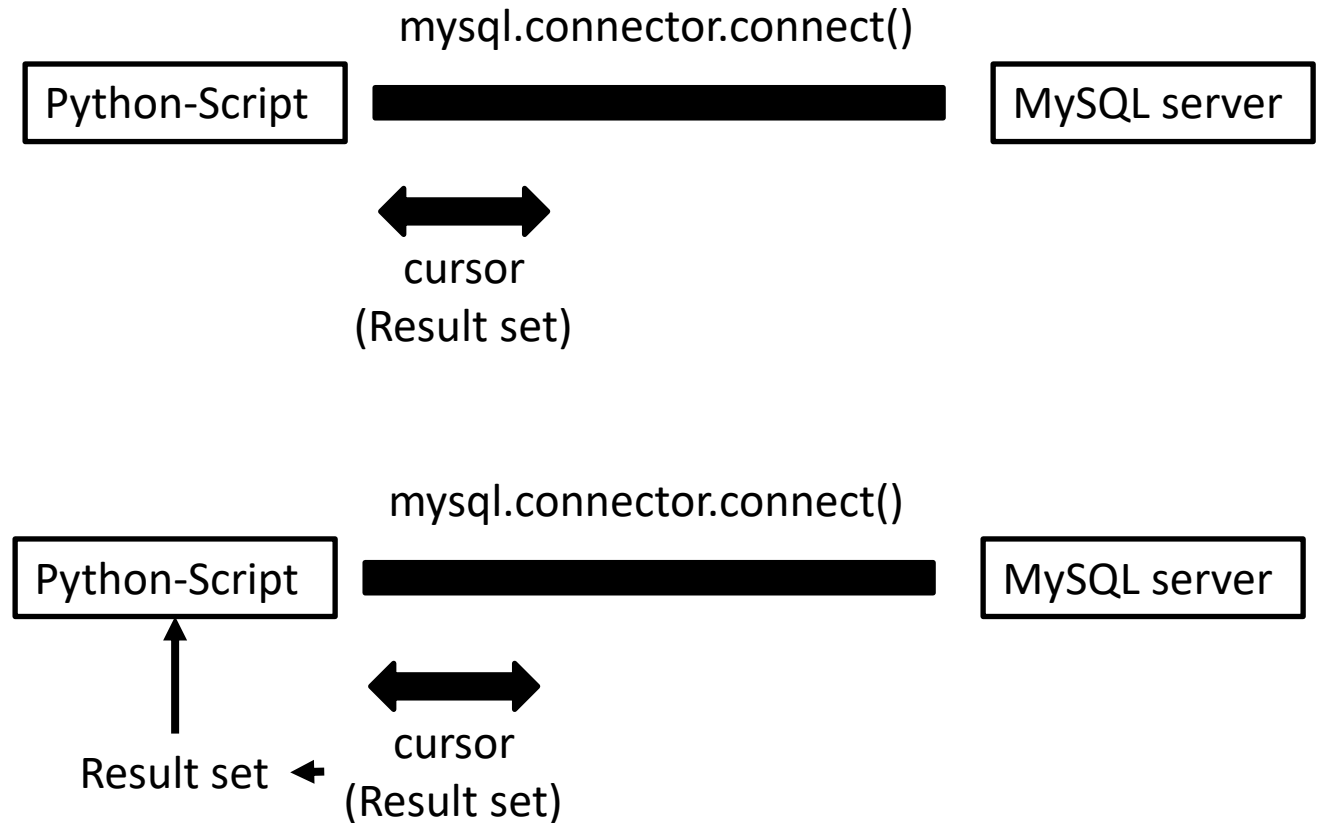
- The variable records is now a list, which contains multiple tuples
- One tuple represents one row of the result set of our SELECT statement, and the number of elements in each tuple represents the number columns in our result set
- So for the simple example we saw records looks like this:

```
[(1, 'Bacteria'), (2, 'Archaea'), (3, 'Protozoa'), (4, 'Chromista'), (5, 'Plantae'), (6, 'Fungi'), (7, 'Animalia'), (8, 'Viruses')]
```

- We learned that the cursor hands over the SQL statement to the server
- The server executes the statement and hands over the produced result set to the cursor



- Now the cursor goes back to our Python script
- By using `fetchall()` we now tell our cursor to hand over the result set to our python script



- There is no limit to our SELECT statements, that means we can use everything we learned during the SQL part here as well
- For example we can add conditions using WHERE
- We can take only some columns from our tables to be shown in the result set
- We can combine conditions using AND / OR as well as negating conditions with NOT
- We have access to the statements IN, LIKE and BETWEEN
- We can use aggregate functions like COUNT, MAX, MIN, AVG or SUM
- We can use sub queries

- When handing over a SELECT statement to our database with the help of the execute function, we ALWAYS need to use fetchall() afterwards to unload our cursor, so that we can hand over a new SELECT statement to it afterwards
- The same is true for SHOW statements as well
- That means everytime you use SELECT or SHOW in execute you need to use fetchall() afterwards
- For all other statements we don't need to use fetchall() afterwards

- You created many different SELECT statements for Exercise 1 up until Exercise 16
- Pick at least eight of them and run them using a Python script with the help of the `mysql-connector-python`
- Please print out the results for each of them

CREATE TABLE

- When working with MySQL in Python we are also able to create tables, just as we could do in our Workbench
- We can use the exact same syntax for our CREATE TABLE statements as we could when working in the workbench
- That means all datatypes as well as all Constraints follow the exact same rules we discussed during the normal SQL part

- To use the CREATE TABLE statement we need to use the execute() function of our cursor again:

```
cursor.execute("""CREATE TABLE boxes (ID int NOT NULL Auto_Increment,  
Type varchar(255), Subtype varchar(255), manufacturer varchar(255),  
PRIMARY KEY (ID)) """)
```

- As you can see I used triple quotes here instead of single quotes
- This allows me to write the very long CREATE TABLE statement in multiple lines without breaking the string
- This can be especially useful when you're dealing with very long SQL statements

- Write a Python program that creates the Tables below, don't forget to add all necessary constraints, especially the Primary and Foreign Keys
- Just like with Exercise 20 the column names are in brackets after the Table name:
 - ▶ publishers (ID, Publisher_Name, City, Address, EMail, Telephone)
 - ▶ books (ID, Book_Title, ISBN, Publication_Date, Publisher_ID, Page_Count, Price, Original_Language)
 - ▶ authors (ID, Last_Name, Given_Name, Title, Birthdate, Birthplace)
 - ▶ wrote (Book_ID, Author_ID)

INSERT INTO

- Filling tables with data by using a Python script is done by using the SQL statement INSERT INTO
- In general it follows the same principles we learned when talking about this in our general SQL part
- However we are able to use loops, functions and lists from our Python script to automate the whole process in our script

- Generally we can use the `execute()` function of our cursor in combination with an `INSERT INTO` statement to add data to our tables:

```
cursor.execute("INSERT INTO boxes (Type,Subtype,Manufacturer) VALUES  
( 'Satin Cube', 'Light Blue', 'Ultra Pro' )")
```

- When working with `INSERT INTO` like this, don't forget to add quotes around the text values of your table and use different kinds of quotes around the whole statement („) and the text values (‘)
- After the `execute()` function we have to commit the changes we made to our table to the server by using the `commit()` function of our connection:

```
connection.commit()
```

- Of course adding data to our tables like this doesn't make so much sense when we want to automate the task
- In these cases it's better to define a function that handles the process
- This function needs one parameter for each column you want to fill with the INSERT INTO statement later on:

```
def insert_into_table(arg_type, arg_sub, arg_manu) :
```

- In your function you can now define a so called “query string” but instead of writing real values you use the place holder %s instead:

```
query = "INSERT INTO boxes (Type,Subtype,Manufacturer) VALUES (%s, %s,  
%s) "
```

- Afterwards you need to create a tuple out the parameters you defined in your signature
- Make sure that the order in this tuple matches the order of the column names in your “query string”:

```
record = (arg_type,arg_sub,arg_manu)
```

- Now you can use the `execute()` function of your cursor and hand over the “query string” and the generated tuple to it as arguments
- Be aware that the “query string” always needs to be the first argument and the tuple the second argument:

```
cursor.execute(query, record)
```

- Afterwards you need to commit your changes again to your server:

```
connection.commit()
```

- At the end your whole function should look like this:

```
def insert_into_table(arg_type,arg_sub,arg_manu):  
    query = "INSERT INTO boxes (Type,Subtype,Manufacturer) VALUES (%s,  
%s, %s) "  
    record = (arg_type,arg_sub,arg_manu)  
    cursor.execute(query,record)  
    connection.commit()
```

- Now you can execute the function with a simple call back:

```
insert_into_table('Squire XL','Blue','Game Genic')
```

- Once you have this function you're able to make use of a for loop in combination with the zip function for example
- Let's say you have the following three lists:

```
Types = ['Motiv Box', 'Motiv Box', 'Gaming Box']
```

```
Subtypes = ['Strefan', 'Henzie', 'Purple']
```

```
Manufacturers = ['Ultra Pro', 'Ultra Pro', 'Dragon Shield']
```

- Now you can create a loop with zip to iterate over all three of them at the same time:

```
for atype, asub, amanu in zip(Types, Subtypes, Manufacturers):
```

- Inside your loop you can now again execute your function with a simple call back, so that your loop looks as follows at the end:

```
for atype, asub, amanu in zip(Types, Subtypes, Manufacturers):  
    insert_into_table(atype, asub, amanu)
```

- This loop allows you to insert many rows of data into your tables with a relative small amount of code lines
- This is especially useful when working with data from external applications for example

- Write a Python script that allows you to fill the tables you created in Exercise 29
- Create one function for each of your tables
- Call each function at least twice to fill the tables with at least two rows of data

1. <https://pynative.com/python-mysql-database-connection/> (Last edited: 09.03.2021, Last visited: 02.05.2023)
2. <https://pynative.com/python-mysql-select-query-to-fetch-data/> (Last edited: 09.03.2021, Last visited: 02.05.2023)
3. <https://pynative.com/python-mysql-insert-data-into-database-table/> (Last edited: 09.03.2021, Last visited: 02.05.2023)